

Membrane Neuron Simulator

Formulation and implementation of a static solver for fluid-driven membrane neurons

W. J. van Vuuren

Technical reference accompanying the source code

<https://github.com/vanvuurenwerner2305-blip/MechanicalNeuronSimulation>

Abstract

This document describes the mathematics and algorithms of the Membrane Neuron Simulator, the software used to simulate the mechanical neurons of the accompanying thesis. A mechanical neuron consists of thin rubber membranes that separate fluid chambers and deform against rigid parts. The software imports such a device from CAD, models the membranes as geometrically nonlinear shells, the chambers as volume-dependent pressure laws and the rigid parts as contact obstacles, and solves for static equilibrium by minimising a total potential energy with a safeguarded Newton method. From a sweep over the input pressures it then back-infers the neuron equation, in the form of the mechanical weights of the neuron model, as low-degree piecewise polynomials.

None of the ingredients is new: the membrane and bending elements, the pressure loads, the penalty contact and the Newton globalisation are standard techniques from the literature, cited where they are used. The purpose of this document is to state exactly which formulation the code implements, so that the results in the thesis can be reproduced and checked.

Contents

1	Scope and overview	3
2	Equilibrium as energy minimisation	4
3	Membrane energy	4
4	Bending energy	5
5	Fluid chambers	6
5.1	Volume change from the membranes alone	6
5.2	Pressure laws	6
5.3	Potential, load and tangent	6
5.4	Load path	7
6	Contact with rigid bodies	7
6.1	Exact signed distance	7
6.2	Penalty energy	7
7	Contact between membranes	8
8	Assembly and exact derivatives	9
9	Newton iteration	9
9.1	Line search on the energy	10
9.2	Second-order volume correction	10

10 Load stepping	10
11 From CAD to model	10
12 Parameter sweeps	11
13 Back-inferring the neuron equation	12
13.1 Input paths	12
13.2 Secant and tangent weights	12
13.3 Sensitivity of the activation pressure	13
13.4 Piecewise polynomial weights	13
13.5 Bias of non-neutral paths	13
14 Verification	15
15 Limitations	15

1 Scope and overview

The simulator answers one question: *given a device and the pressures in its input chambers, what is the static equilibrium shape of its membranes, and what are the resulting pressures in its sealed chambers?* The device is built in CAD (Fusion 360 in practice), exported as a STEP file, and every solid body is given one of the following roles:

- **Membrane:** a thin solid with membrane stiffness only;
- **Shell:** a thin solid with membrane and bending stiffness;
- **Rigid body:** an obstacle that the membranes cannot penetrate;
- **Fluid chamber:** a solid that fills a fluid space, with a pressure model (constant-pressure input, sealed ideal gas, sealed liquid, or vent to the surroundings);
- **Ignore.**

Chamber walls are never meshed. Rigid bodies are meshed as closed surfaces, and membranes and shells are replaced by triangulated mid-surfaces.

The whole analysis is *quasi-static*. Nothing depends on time: there is no inertia, no viscosity and no fluid flow. Every chamber is a lumped volume with one uniform pressure. This matches how the neuron is analysed in the thesis, where the input pressures are held at steady state and the neuron’s output is the equilibrium pressure of its sealed activation chamber.

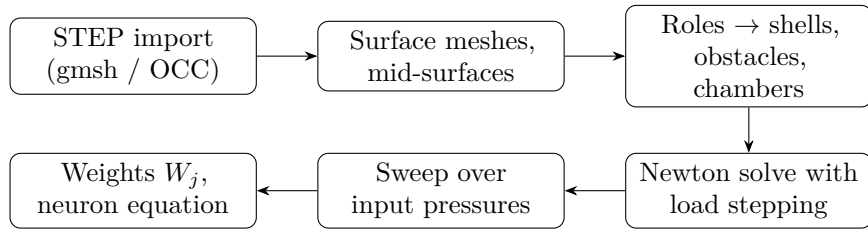


Figure 1: Pipeline of the simulator (Sections 11, 2–10 and 13).

Units and conventions. The application works in millimetres, newtons and megapascals (so energies are in N mm), and it reports pressures in kPa. All pressures are *gauge* pressures: the surroundings are at 0. A membrane face that touches no chamber therefore sees zero pressure. The solver core (`membrane_sim`) itself has no units; it works in whatever consistent system the caller uses.

Code map. Table 1 links the sections of this document to the source files.

File	Contents	Section
<code>membrane_sim/shell.py</code>	membrane and bending elements	3, 4
<code>membrane_sim/fluid.py</code>	chamber volumes and pressure laws	5
<code>membrane_sim/contact.py</code>	rigid obstacles, signed distance	6
<code>membrane_sim/shell_contact.py</code>	contact between membranes	7
<code>membrane_sim/solver.py</code>	assembly and Newton iteration	8, 9
<code>membrane_sim/environment.py</code>	load stepping	10
<code>membrane_sim/characterise.py</code>	weights and neuron equation	13
<code>app/cad.py</code> , <code>app/builder.py</code>	CAD import, meshing, coupling	11
<code>app/sweep.py</code>	parameter sweeps	12

Table 1: Where each part of the formulation lives in the code.

2 Equilibrium as energy minimisation

Let $\mathbf{u} \in \mathbb{R}^n$ collect all unknowns: the current position $\mathbf{x}_a \in \mathbb{R}^3$ of every mesh node a of every shell and, for shells with bending, one rotation ψ_e per mesh edge (Section 4). The rest positions are $\mathbf{X}_a$. Nodes on a clamped boundary are fixed at their rest positions and removed from the unknowns.

Every force in the model is conservative, so the static equilibrium is a stationary point of the total potential energy

$$\Pi(\mathbf{u}; \lambda) = \sum_{\text{shells}} \left(U_{\text{mem}}(\mathbf{u}) + U_{\text{bend}}(\mathbf{u}) \right) + E_{\text{rigid}}(\mathbf{u}) + E_{\text{sheet}}(\mathbf{u}) - \sum_{\text{chambers } v} \Phi_v(\Delta V_v(\mathbf{u}); \lambda), \quad (1)$$

where U_{mem} and U_{bend} are the strain energies of the shells, E_{rigid} and E_{sheet} are penalty energies for contact with rigid bodies and between membranes, ΔV_v is the volume change of chamber v , Φ_v is the work done by its fluid, and $\lambda \in [0, 1]$ is the load factor used to ramp the pressures. The solver finds a point where

$$\mathbf{R}(\mathbf{u}) = \frac{\partial \Pi}{\partial \mathbf{u}} = \mathbf{0}, \quad (2)$$

using the tangent $\mathbf{K} = \partial^2 \Pi / \partial \mathbf{u}^2$, and it uses Π itself as the merit function of its line search (Section 9). Stable equilibria are local minima of Π . Because the formulation is variational, $\mathbf{K}$ is symmetric.

3 Membrane energy

Each shell is a mesh of three-node triangles with constant strain (CST). For a triangle with rest vertices $\mathbf{X}_0, \mathbf{X}_1, \mathbf{X}_2$, an orthonormal in-plane basis is built from its rest normal $\mathbf{N}_0$:

$$\mathbf{e}_1 = \frac{\mathbf{X}_1 - \mathbf{X}_0}{\|\mathbf{X}_1 - \mathbf{X}_0\|}, \quad \mathbf{e}_2 = \mathbf{N}_0 \times \mathbf{e}_1, \quad \mathbf{D}_m = \begin{bmatrix} \mathbf{e}_1 \cdot (\mathbf{X}_1 - \mathbf{X}_0) & \mathbf{e}_1 \cdot (\mathbf{X}_2 - \mathbf{X}_0) \\ \mathbf{e}_2 \cdot (\mathbf{X}_1 - \mathbf{X}_0) & \mathbf{e}_2 \cdot (\mathbf{X}_2 - \mathbf{X}_0) \end{bmatrix}. \quad (3)$$

With the current edge matrix $\mathbf{D}_s = [\mathbf{x}_1 - \mathbf{x}_0, \mathbf{x}_2 - \mathbf{x}_0] \in \mathbb{R}^{3 \times 2}$, the surface deformation gradient and the right Cauchy–Green tensor are

$$\mathbf{F} = \mathbf{D}_s \mathbf{D}_m^{-1} \in \mathbb{R}^{3 \times 2}, \quad \mathbf{C} = \mathbf{F}^\top \mathbf{F} \in \mathbb{R}^{2 \times 2}. \quad (4)$$

This is exact for arbitrarily large displacements and rotations [2]. With A_0 the rest area and t the thickness, two materials are available.

Incompressible neo-Hookean (default, rubber). For an incompressible sheet in plane stress, the through-thickness stretch is $\lambda_3 = 1/J$ with $J^2 = \det \mathbf{C}$ (the area stretch). Substituting it into the three-dimensional invariant $I_1 = \text{tr} \mathbf{C} + \lambda_3^2$ gives the energy per element

$$U_{\text{mem}}^e = A_0 \frac{\mu t}{2} \left(\text{tr} \mathbf{C} + \frac{1}{\det \mathbf{C}} - 3 \right) + T A, \quad \mu = \frac{E}{3}, \quad (5)$$

where $\mu = E/3$ is the shear modulus of an incompressible material ($\nu = 1/2$). The thickness change is eliminated analytically, so no extra unknown is needed.

St. Venant–Kirchhoff. On the Green–Lagrange strain $\mathbf{E} = \frac{1}{2}(\mathbf{C} - \mathbf{I})$, with the plane-stress Lamé constant,

$$U_{\text{mem}}^e = A_0 t \left(\frac{\bar{\lambda}}{2} (\text{tr} \mathbf{E})^2 + \mu \mathbf{E} : \mathbf{E} \right) + T A, \quad \mu = \frac{E}{2(1 + \nu)}, \quad \bar{\lambda} = \frac{E\nu}{1 - \nu^2}. \quad (6)$$

This model is only meaningful for small strains. It is kept for comparison with linear plate theory.

Pre-tension. In both cases TA adds an optional isotropic in-plane pre-tension T (force per length) times the *current* element area $A = \frac{1}{2}\|(\mathbf{x}_1 - \mathbf{x}_0) \times (\mathbf{x}_2 - \mathbf{x}_0)\|$. It acts like a surface tension. It is zero by default.

Remark. A membrane without pre-tension or bending stiffness has no transverse stiffness in its flat state: the tangent is singular at the start of every solve. The load stepping and the damped Newton steps of Section 9 deal with this. It is also the physical reason why the weights of Section 13 behave singularly near zero pressure difference: a clamped circular membrane under pressure p (Hencky’s problem [8]) deflects as $w \propto p^{1/3}$, so its displaced volume per unit pressure behaves like $p^{-2/3}$.

4 Bending energy

The **Shell** role adds bending stiffness through a geometrically nonlinear version of Morley’s constant-moment triangle [12], written with mid-edge normals as in discrete shell models [5, 6]. Every mesh edge e carries one extra unknown, the rotation ψ_e of its mid-edge normal. The first attempt used an “average face normal” hinge without ψ_e . That model’s bending response depended on the orientation of the mesh, and the ψ_e unknowns remove that dependence. A test checks this (`test_bending_energy_is_independent_of_mesh_orientation`).

Edge rotations. For triangle f with current normal $\mathbf{n}_1$ and edge k (opposite vertex k , running from vertex i to vertex j with unit direction $\hat{\mathbf{e}}$), the signed angle between $\mathbf{n}_1$ and a second normal $\mathbf{n}_2$ about the edge is

$$\theta_k = \text{atan2}((\mathbf{n}_1 \times \mathbf{n}_2) \cdot \hat{\mathbf{e}}, \mathbf{n}_1 \cdot \mathbf{n}_2). \quad (7)$$

The rotation ϕ_k of the mid-edge normal relative to face f depends on the edge type:

$$\phi_k = \begin{cases} \frac{1}{2}\theta_k + s_k \psi_e, & \text{interior edge, } \mathbf{n}_2 = \text{normal of the neighbouring face,} \\ \theta_k, & \text{clamped boundary edge, } \mathbf{n}_2 = \mathbf{N}_0 \text{ (rest normal),} \\ \psi_e, & \text{free (hinged) boundary edge.} \end{cases} \quad (8)$$

Here $s_k = \pm 1$ records the direction in which face f traverses the edge, so the two faces that share an interior edge see the same mid-edge normal. Here θ_k is the full dihedral angle, so $\frac{1}{2}\theta_k$ puts the mid-edge normal halfway between the two faces, and ψ_e lets it rotate further. A clamped edge is one whose two end nodes are both fixed while the shell’s boundary rotation is set to “clamped”. Its mid-edge normal is held at the rest normal, and its ψ_e is removed from the unknowns.

Curvature and energy. The face has the constant curvature tensor

$$\boldsymbol{\kappa} = \frac{1}{A_0} \sum_{k=1}^3 l_k \phi_k \mathbf{m}_k \otimes \mathbf{m}_k, \quad (9)$$

with rest edge lengths l_k and in-plane outward edge normals $\mathbf{m}_k$ taken from the rest state. With the change of curvature $\Delta\boldsymbol{\kappa}$ measured from the rest state ($\Delta\phi_k = \phi_k - \phi_k^0$, so curved CAD geometry is stress-free), the energy of a linear-elastic plate,

$$U_{\text{bend}}^e = A_0 \frac{D}{2} \left(\nu (\text{tr } \Delta\boldsymbol{\kappa})^2 + (1 - \nu) \Delta\boldsymbol{\kappa} : \Delta\boldsymbol{\kappa} \right), \quad D = \frac{Et^3}{12(1 - \nu^2)}, \quad (10)$$

becomes a quadratic form in $\Delta\boldsymbol{\phi}$. With $\text{tr}(\mathbf{m}_i \otimes \mathbf{m}_i) = 1$ and $(\mathbf{m}_i \otimes \mathbf{m}_i) : (\mathbf{m}_j \otimes \mathbf{m}_j) = (\mathbf{m}_i \cdot \mathbf{m}_j)^2$,

$$U_{\text{bend}}^e = \Delta\boldsymbol{\phi}^\top \mathbf{M} \Delta\boldsymbol{\phi}, \quad M_{ij} = \frac{D}{2A_0} l_i l_j \left(\nu + (1 - \nu)(\mathbf{m}_i \cdot \mathbf{m}_j)^2 \right). \quad (11)$$

$\mathbf{M}$ depends only on the rest geometry and is precomputed. All the geometric nonlinearity sits in $\phi_k(\mathbf{u})$. For the neo-Hookean material $\nu = \frac{1}{2}$, so $D = Et^3/9$.

5 Fluid chambers

5.1 Volume change from the membranes alone

A chamber is bounded partly by rigid walls and partly by membranes. Only its *change* in volume matters, and it can be computed from the membranes alone. Every membrane is clamped along its boundary, so the rest mesh and the deformed mesh of the same membrane together close a (possibly self-intersecting) volume, and that volume is exactly the volume the membrane has swept. Using signed cone volumes about any fixed point $\mathbf{o}$,

$$V_{\text{cone}}(\mathbf{x}) = \frac{1}{6} \sum_{\text{faces}} (\mathbf{x}_0 - \mathbf{o}) \cdot ((\mathbf{x}_1 - \mathbf{o}) \times (\mathbf{x}_2 - \mathbf{o})), \quad (12)$$

the volume change of chamber v is

$$\Delta V_v(\mathbf{u}) = \sum_{\text{shells } s \text{ on } v} \sigma_{s,v} (V_{\text{cone}}(\mathbf{x}_s) - V_{\text{cone}}(\mathbf{X}_s)), \quad (13)$$

where $\sigma_{s,v} = +1$ if the shell's normal points out of the chamber (the chamber lies behind the shell and pushes it along its normal) and -1 otherwise. The result does not depend on $\mathbf{o}$ or on the rigid walls, which is why chamber walls never need a mesh. The code uses the centroid of the rest mesh for $\mathbf{o}$, which limits round-off.

ΔV_v is a cubic polynomial in the node coordinates. Its gradient is $\mathbf{g}_v = \partial \Delta V_v / \partial \mathbf{u}$, which per face is $\frac{1}{6}(\mathbf{x}_1 \times \mathbf{x}_2, \mathbf{x}_2 \times \mathbf{x}_0, \mathbf{x}_0 \times \mathbf{x}_1)$ (relative to $\mathbf{o}$). Its Hessian is constant per face.

5.2 Pressure laws

Each chamber has a pressure law $P_v(\Delta V)$ with rest pressure P_0 :

$$\text{constant (input, vent)} : P = P_0 \quad (\text{vent: } P_0 = 0), \quad (14)$$

$$\text{sealed liquid} : P = P_0 - K (\Delta V + V_{\text{ex}}), \quad K = \frac{100 s}{V_{\text{liq}}}, \quad V_{\text{ex}} = V_{\text{body}} - V_{\text{liq}}, \quad (15)$$

$$\text{sealed ideal gas} : P = (P_{\text{atm}} + P_0) \frac{V_g}{V_g + \Delta V} - P_{\text{atm}}. \quad (16)$$

For the liquid, the user gives a stiffness s in pressure per percent of volume change and a liquid volume V_{liq} , which defaults to the volume of the CAD body. If the chamber holds less liquid than its body volume ($V_{\text{ex}} > 0$), the liquid is in suction and pulls the membranes in; if it holds more, it inflates them. Water corresponds to $s \approx 22\,000$ kPa/%. Above about 1000 kPa/% the results hardly change, but the Newton iteration gets slower.

For the gas, Boyle's law is applied to the gas share only: $V_g = (1 - f)(V_{\text{body}} + V_{\text{ghost}})$, where f is the liquid fraction (at most 99%) and V_{ghost} is an optional extra gas volume outside the modelled chamber, such as tubing. Any liquid in the chamber is incompressible, so every volume change goes into the gas. A state with $V_g + \Delta V \leq 0$ gets infinite energy and is rejected by the line search. A user supplied, differentiable law $P(\Delta V; P_0)$ is also supported.

5.3 Potential, load and tangent

The fluid's work is $\Phi_v(\Delta V) = \int_0^{\Delta V} P_v(s) ds$. In closed form,

$$\Phi^{\text{liq}} = P_0 \Delta V - K \left(V_{\text{ex}} \Delta V + \frac{1}{2} \Delta V^2 \right), \quad \Phi^{\text{gas}} = (P_{\text{atm}} + P_0) V_g \ln \frac{V_g + \Delta V}{V_g} - P_{\text{atm}} \Delta V, \quad (17)$$

and a custom law is integrated with 16-point Gauss–Legendre quadrature. Differentiating $-\Phi_v(\Delta V_v(\mathbf{u}))$ gives the pressure load and its tangent:

$$\frac{\partial(-\Phi_v)}{\partial \mathbf{u}} = -P_v \mathbf{g}_v, \quad \frac{\partial^2(-\Phi_v)}{\partial \mathbf{u}^2} = -P_v \frac{\partial^2 \Delta V_v}{\partial \mathbf{u}^2} - P'_v \mathbf{g}_v \mathbf{g}_v^\top. \quad (18)$$

The force $P_v \mathbf{g}_v$ is exactly the follower pressure load, which acts normal to the *deformed* surface. The first tangent term is the usual load-stiffness term, and it is symmetric because the load has a potential. The second term couples every node of a chamber to every other node. It is a dense rank-one matrix, and Section 8 handles it without forming it. Since $P'_v \leq 0$ for every sealed chamber, $c_v = -P'_v \geq 0$ is a volumetric stiffness.

5.4 Load path

A fresh solve ramps every chamber from zero pressure: the solver uses $\lambda P_v(\Delta V)$ (and likewise for the slope and potential). A warm-started solve (Section 12) starts from a converged state with rest pressure P_0^{old} and blends to the new rest pressure,

$$P_v^\lambda(\Delta V) = (1 - \lambda) P_v(\Delta V; P_0^{\text{old}}) + \lambda P_v(\Delta V; P_0). \quad (19)$$

6 Contact with rigid bodies

6.1 Exact signed distance

A rigid body is a closed, outward-oriented triangle mesh. For a point $\mathbf{p}$, the closest point $\mathbf{q}$ on each candidate triangle is found with the region tests of Ericson [3], which also report whether $\mathbf{q}$ lies on a face, an edge or a vertex. The sign of the distance comes from the *angle-weighted pseudo-normal* $\mathbf{N}_q$ of that feature [1]: the face normal on a face, the normalised sum of the two adjacent face normals on an edge, and the sum of the incident face normals weighted by their angles at a vertex. For a closed mesh this sign is exact:

$$d(\mathbf{p}) = \text{sign}((\mathbf{p} - \mathbf{q}) \cdot \mathbf{N}_q) \|\mathbf{p} - \mathbf{q}\|, \quad \nabla d = \text{sign}(\cdot) \frac{\mathbf{p} - \mathbf{q}}{\|\mathbf{p} - \mathbf{q}\|}. \quad (20)$$

$d < 0$ inside the solid. An *inverted* obstacle (a container whose inside is the free space) flips the sign, and several obstacles combine by taking the minimum. The Hessian of the distance is needed for a consistent tangent:

$$\nabla^2 d = \text{sign}(\cdot) \frac{1}{|d|} \begin{cases} \mathbf{0}, & \mathbf{q} \text{ on a face,} \\ \mathbf{I} - \hat{\mathbf{u}}\hat{\mathbf{u}}^\top - \hat{\mathbf{t}}\hat{\mathbf{t}}^\top, & \mathbf{q} \text{ on an edge with direction } \hat{\mathbf{t}}, \\ \mathbf{I} - \hat{\mathbf{u}}\hat{\mathbf{u}}^\top, & \mathbf{q} \text{ at a vertex,} \end{cases} \quad (21)$$

with $\hat{\mathbf{u}} = (\mathbf{p} - \mathbf{q})/\|\mathbf{p} - \mathbf{q}\|$. On a face the Hessian is zero because the face is flat.

Broad phase. A nearest-vertex distance gives an upper bound on the closest distance, and triangles whose bounding box is farther than that bound are skipped. Between Newton iterations the candidate triangles of each node are cached in Verlet lists [17]. At a rebuild, each node keeps every triangle within its current distance plus $2\delta_s$ (the skin δ_s is the larger of the contact reach and 1 % of the model size), and the list is rebuilt as soon as any node has moved more than δ_s . Until then the true closest triangle is guaranteed to be in the list, so the cached query gives *exactly* the result of a full search. A test checks this (`test_cached_contact_queries_match_the_full_search`).

6.2 Penalty energy

Each free node a of a shell keeps a clearance r_a from the obstacles: half the shell thickness, reduced to the rest distance where the CAD geometry already starts closer than that. The contact energy is a one-sided quadratic penalty,

$$E_{\text{rigid}} = \sum_a \frac{1}{2} k A_a \min(d(\mathbf{x}_a) - r_a, 0)^2, \quad (22)$$

with A_a the lumped nodal area (one third of the adjacent rest triangle areas). Weighting by area makes k a pressure per unit penetration, independent of the mesh. For an active node with gap $\gamma_a = d - r_a < 0$, the force and tangent are

$$\frac{\partial E}{\partial \mathbf{x}_a} = k A_a \gamma_a \nabla d, \quad \frac{\partial^2 E}{\partial \mathbf{x}_a^2} = k A_a \left(\nabla d \nabla d^\top + \gamma_a \nabla^2 d \right). \quad (23)$$

Choice of k . A penalty always allows some penetration. At a contact pressure p the penetration is about p/k . The application chooses

$$k = \frac{\max(p_{\text{ref}}, 1 \text{ kPa})}{0.05 t_{\min}}, \quad (24)$$

where p_{ref} is the highest chamber pressure in the solve or sweep and $t_{\min}$ is the thinnest part, so the penetration stays around 5 % of the thinnest thickness.

Clamp walls. A membrane in CAD is usually clamped between rigid frame parts, so the free nodes next to its fixed boundary start right at a rigid wall. These nodes sit on the on/off kink of the penalty, and they made the Newton iteration cycle. Free nodes that share a triangle with a fixed node and start within $1.25 r_a$ of an obstacle are therefore excluded from rigid contact.

7 Contact between membranes

Membranes can also touch each other (not themselves). Each sheet is a mid-surface with thickness t , so node a of sheet A touches sheet B when it comes closer than $h = \frac{1}{2}t_A + \frac{1}{2}t_B$ to B 's mid-surface. As for rigid contact, h is reduced to the rest distance where the sheets already start closer, for example where they meet at a frame. For every node–triangle pair within range, with d the distance from the node to the closest point of that triangle and penetration $\pi = h - d$,

$$E_{\text{sheet}} = \sum_{\text{pairs}} k A_a \varphi(\pi), \quad \varphi(\pi) = \begin{cases} 0, & \pi \leq 0, \\ \frac{\pi^3}{6\delta}, & 0 < \pi < \delta, \\ \frac{\pi^2}{2} - \frac{\delta\pi}{2} + \frac{\delta^2}{6}, & \pi \geq \delta, \end{cases} \quad \delta = 0.25 h. \quad (25)$$

The function φ is the ordinary quadratic penalty with its stiffness ramped up over the first δ of penetration. φ , φ' and φ'' are continuous at $\pi = 0$ and at $\pi = \delta$ (the three values at δ are $\delta^2/6$, $\delta/2$ and 1 from either side), so a pair coming into contact does not make the tangent jump. *All* triangles within range contribute, not only the closest. With the closest only, the force jumps from one triangle's vertices to its neighbour's when a node slides across an edge, and Newton cycled. Both directions (A 's nodes on B and B 's nodes on A) are included. The gradient and Hessian of each pair's energy with respect to its twelve coordinates are exact (Section 8).

No tunnelling. A penalty can be stepped over: one large step can carry a node straight through the other sheet to where the penalty is zero again. As in the continuous collision handling of Incremental Potential Contact [10], every line-search step is therefore bounded conservatively. A node at distance d from a triangle can only reach it if the two approach each other by d , and in a step $\alpha \Delta \mathbf{u}$ they approach by at most $\alpha(\|\Delta \mathbf{x}_a\| + \max_{b \in \text{tri}} \|\Delta \mathbf{x}_b\|)$. The step fraction is limited to

$$\alpha \leq \min_{\text{pairs}} \frac{0.9 d}{\|\Delta \mathbf{x}_a\| + \max_b \|\Delta \mathbf{x}_b\|}, \quad (26)$$

so no pair closes more than 90 % of its current distance in one step. Pairs whose bounding-box distance already satisfies the bound are skipped without computing the exact distance.

8 Assembly and exact derivatives

Every energy above is written as a function of one element’s coordinates: a membrane triangle (9 values), a bending stencil (the triangle, its three opposite nodes and three edge rotations: 21 values), a pressure face (9), or a sheet-contact pair (12). The gradient and Hessian of each element energy are obtained by automatic differentiation with `torch.func` (`grad`, `hessian`, vectorised over the elements with `vmap`) [14]. The residual and tangent are therefore the *exact* derivatives of the discrete energy, which gives quadratic Newton convergence without hand-derived tangent matrices. A test compares them with finite differences of Π (`test_residual_is_energy_gradient_and_tangent_is_its_jacobian`). Everything is computed in double precision.

The element contributions are assembled into a sparse matrix, and the rows and columns of fixed unknowns are dropped. The chamber terms $P'_v \mathbf{g}_v \mathbf{g}_v^\top$ of (18) are kept apart. Writing $\mathbf{U} = [\mathbf{g}_1, \dots, \mathbf{g}_m]$ for the volume gradients of the m chambers with $P'_v \neq 0$ and $\mathbf{C} = \text{diag}(c_v)$, the tangent is

$$\mathbf{K}_{\text{full}} = \mathbf{A} + \mathbf{U} \mathbf{C} \mathbf{U}^\top, \quad (27)$$

where $\mathbf{A}$ is sparse. Linear systems are solved with a sparse LU factorisation of $\mathbf{A}$ (SuperLU through SciPy [18]) and the Sherman–Morrison–Woodbury identity [7]:

$$\mathbf{K}_{\text{full}}^{-1} \mathbf{b} = \mathbf{y} - \mathbf{Z} \left(\mathbf{C}^{-1} + \mathbf{U}^\top \mathbf{Z} \right)^{-1} \mathbf{U}^\top \mathbf{y}, \quad \mathbf{y} = \mathbf{A}^{-1} \mathbf{b}, \quad \mathbf{Z} = \mathbf{A}^{-1} \mathbf{U}. \quad (28)$$

This needs only m extra back-substitutions and one small $m \times m$ solve.

9 Newton iteration

At a fixed load factor λ , the solver iterates from the current state until

$$\|\mathbf{R}\| \leq \tau_a + \tau_r \max(\|\mathbf{f}_{\text{int}}\|, \|\mathbf{f}_{\text{pres}}\|, \|\mathbf{f}_{\text{cont}}\|), \quad (29)$$

with defaults $\tau_r = 10^{-8}$ and $\tau_a = 10^{-12}$. The reference is the largest of the internal, pressure and contact force norms (free unknowns only). An iteration also counts as converged when the undamped Newton step is below 10^{-10} times the model size (round-off floor). Contact energies are only C^1 where a closest point crosses a triangle edge, which puts a floor under the attainable residual. So when the residual has not halved for three iterations and is within 1000 times the tolerance, the iteration also stops as converged.

Each iteration tries the following steps in order, until one decreases the energy:

1. **Newton step with second-order correction.** Solve $\mathbf{K}_{\text{full}} \mathbf{s} = -\mathbf{R}$ with (28) and search along $\mathbf{s}$, correcting every trial point as described below.
2. **Levenberg–Marquardt steps.** If the Newton step fails, for example because the tangent is indefinite or singular (a slack membrane, or a limit point), damp it: $(\mathbf{A} + \mu \bar{a} \mathbf{I} + \mathbf{U} \mathbf{C} \mathbf{U}^\top) \mathbf{s} = -\mathbf{R}$, with $\bar{a}$ the mean absolute diagonal of $\mathbf{A}$. After a failure $\mu \leftarrow \max(10\mu, 10^{-6})$, for up to four attempts. When the first attempt of an iteration is accepted with a full step ($\alpha = 1$), μ is reduced tenfold, or set to zero below 10^{-9} . μ carries over between iterations and load steps, so the first attempt is the undamped Newton step only once the damping has died out.
3. **Gradient step.** As a last resort, search along the Jacobi-preconditioned steepest descent direction $\mathbf{s} = -\mathbf{R} \oslash (|\text{diag } \mathbf{A}| + (\mathbf{U} \circ \mathbf{U}) |\mathbf{c}| + 10^{-8} \bar{a})$, with up to 40 backtracking steps. This always decreases the energy for a small enough step. If it also fails, the Newton solve reports failure.

9.1 Line search on the energy

Every candidate step is first scaled so that no coordinate moves more than 10% of the model size, and it is limited by the no-tunnelling bound of Section 7. A step length α is then accepted under the Armijo condition [13]

$$\Pi(\mathbf{u}_0 + \alpha \mathbf{s}) \leq \Pi(\mathbf{u}_0) + 10^{-4} \alpha \mathbf{R}^\top \mathbf{s}. \quad (30)$$

If the condition fails, the next α is the minimiser of the quadratic through $\Pi(\mathbf{u}_0)$, the slope $\mathbf{R}^\top \mathbf{s}$ and $\Pi(\mathbf{u}_0 + \alpha \mathbf{s})$, kept within $[0.1\alpha, 0.5\alpha]$. It is $\alpha/10$ when the trial energy is infinite (a gas pocket squeezed to zero). A step whose energy change is within round-off of zero is also accepted if it reduces the residual. **The energy is the only merit function.** Mixing in a residual-decrease criterion made the iteration cycle as contact nodes switched in and out of contact.

9.2 Second-order volume correction

Stiff sealed chambers need special treatment. A chamber’s volume is a nonlinear function of the node positions, so along a straight Newton step the actual volume change differs from the linearised one by $O(\alpha^2 \|\mathbf{s}\|^2)$. A nearly incompressible chamber (large c_v) turns this small volume error into a large energy $\frac{1}{2} c_v e_v^2$. The line search then rejects good Newton steps and convergence slows to a crawl. This is the Maratos effect known from constrained optimisation, and the standard remedy is a second-order correction [13]. Each trial point $\mathbf{u}_0 + \alpha \mathbf{s}$ is moved back onto the linearised volumes. With the volume error

$$e_v = \Delta V_v(\mathbf{u}_0 + \alpha \mathbf{s}) - (\Delta V_v(\mathbf{u}_0) + \alpha \mathbf{g}_v^\top \mathbf{s}), \quad (31)$$

the correction is the smallest change, in the norm of $\mathbf{A}$, that removes it to first order:

$$\min_{\boldsymbol{\delta}} \frac{1}{2} \boldsymbol{\delta}^\top \mathbf{A} \boldsymbol{\delta} \quad \text{s.t.} \quad \mathbf{U}^\top \boldsymbol{\delta} = -\mathbf{e} \quad \implies \quad \boldsymbol{\delta} = -\mathbf{Z} \left(\mathbf{U}^\top \mathbf{Z} \right)^{-1} \mathbf{e}, \quad (32)$$

where $\mathbf{Z} = \mathbf{A}^{-1} \mathbf{U}$ is already available from (28). With the correction, stiff and soft chambers converge in a similar number of iterations.

10 Load stepping

The pressures are applied in load steps $\lambda = 0 \rightarrow 1$, with initial increment $1/N$ ($N = 10$ by default). After each converged step the state is stored; the application can play these states back. The increment

- grows by a factor 1.5 (up to $1/N$) when a step converged in at most four iterations;
- is halved, and the step retried from the last converged state, when Newton fails;
- is considered too small below 10^{-3} , at which point the solve stops and reports the last converged load factor.

A snap-through, such as a membrane ballooning past its limit point, has no nearby equilibrium, so cutting the increment does not help. The first time the increment falls below $0.25/N$, the solver makes one long “relaxation” attempt at the target load with five times the usual iteration limit. The energy-descent iteration of Section 9 can then carry the structure to its new equilibrium.

11 From CAD to model

Import. The STEP file is read with gmsh’s OpenCASCADE kernel [4], in millimetres. Every solid is one body. Names come from the product labels, or, for multi-body parts from Fusion 360 whose names gmsh does not read, from the MANIFOLD_SOLID_BREP records of the file.

Meshing. Every body’s boundary is triangulated with gmsh (Frontal–Delaunay, size set at the CAD points). For membranes and shells the element size is the shortest *in-plane* side of the bounding box divided by a number of elements per side (default 10). For solids it is the shortest side divided by that number, but never finer than the diagonal divided by three times that number, so that thin rigid parts do not produce huge meshes. Each closed surface mesh is then oriented outward: the faces of each connected component are made consistent by walking across shared edges, the component is flipped if its signed volume is negative, and a component nested inside an odd number of others is flipped again, because it bounds a cavity. The nesting depth is found with the generalised winding number (below).

Mid-surfaces. A membrane or shell body is replaced by its mid-surface. The body’s largest CAD face is taken as the primary face. From each of its nodes a ray is cast inward along the vertex normal (Möller–Trumbore [11]) to the opposite faces, which measures the local thickness d . The thickness is the median of these distances. Local values outside $[\text{median}/3, 3 \text{ median}]$, or rays that miss, are replaced by the median, and every node is moved inward by $\frac{1}{2}d$. The mid-surface keeps the orientation of the primary face.

Boundary conditions. By default all boundary nodes of the mid-surface are fixed (with clamped rotations for shells). Alternatively, only boundary nodes within $0.75t$ of a rigid body are fixed, which models a membrane clamped by a frame on some edges and free elsewhere.

Chamber coupling. The software must find out which chambers act on which side of which membrane. From the centroid of every mid-surface triangle, a probe point is placed on each side at distance $\frac{1}{2}t + \max(\frac{1}{4}t, 10^{-3})$ along the normal. A probe lies inside a chamber when the generalised winding number of the chamber’s mesh exceeds $\frac{1}{2}$ [9]. The winding number is evaluated as a sum of triangle solid angles [16]:

$$w(\mathbf{p}) = \frac{1}{4\pi} \sum_{\text{tri}} 2 \operatorname{atan2} \left(\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}), abc + (\mathbf{a} \cdot \mathbf{b})c + (\mathbf{a} \cdot \mathbf{c})b + (\mathbf{b} \cdot \mathbf{c})a \right), \quad (33)$$

with $\mathbf{a}, \mathbf{b}, \mathbf{c}$ the triangle vertices relative to $\mathbf{p}$ and a, b, c their lengths. If at least 30 % of the probes on one side are inside, the chamber is coupled to that side of the membrane with the corresponding σ of (13). A chamber detected on both sides has no net effect and is ignored with a warning. A warning is also given when the coverage is below 90 %, because the pressure is then applied to the whole membrane.

Chamber parameters. Constant-pressure chambers are the inputs. Sealed liquid chambers use (15), with V_{body} the CAD volume. Sealed gas chambers use (16) with $P_{\text{atm}} = 101.325 \text{ kPa}$. Vents are constant-pressure chambers at zero.

12 Parameter sweeps

A sweep solves the model on a grid of one or two input-chamber pressures. The first point is a fresh solve. Every following point is warm-started from the previous solution, ramping only the change in rest pressure, with two load steps. If that fails, the point is solved fresh. One penalty stiffness, chosen for the highest pressure in the sweep, is used for all points, so that neighbouring points are consistent. Warm starting is several times faster, because contact is already established. A test checks that it reaches the same state as a fresh solve (`test_warm_start_reaches_the_same_state_as_a_fresh_solve`).

13 Back-inferring the neuron equation

The thesis models the input stage of the neuron as a sealed activation chamber a whose equilibrium pressure is a weighted average of the pressures acting on it:

$$p_a = \frac{\sum_j W_j p_j + W_0 p_0 + B}{\sum_j W_j + W_0}, \quad (34)$$

where W_j is the mechanical weight (compliance) of input j , W_0 the compliance of the activation fluid itself about its rest pressure p_0 , and B a bias volume that is zero unless a path contains a chamber that is not neutral at rest (Section 13.5). In general the weights depend on the pressure differences, $W_j = W_j(p_j - p_a)$, so (34) is implicit in p_a . This section describes how the weights are measured from simulations and fitted.

13.1 Input paths

A weight belongs to an *input path*, not to a single membrane. Everything between an input chamber and the activation chamber is lumped together. This can be one membrane, or, for the bulk-modulus weight mechanism, membrane – sealed weight chamber – membrane. Paths are found from the model’s topology. Starting from every shell on the activation chamber, the far side of the shell is examined:

- no chamber there: the path reaches the surroundings, an *ambient* input at 0;
- a constant-pressure chamber: the path reaches that input;
- another sealed chamber: the search continues through the other shells of that chamber.

Shells that reach the same input form one path. A shell that reaches several inputs gets a combined label.

13.2 Secant and tangent weights

For path j with shells s on the activation chamber, the volume pushed into the activation chamber and the secant weight are

$$\Delta V_j = - \sum_{s \in j} \sigma_{s,a} (V_{\text{cone}}(\mathbf{x}_s) - V_{\text{cone}}(\mathbf{X}_s)), \quad W_j = \frac{\Delta V_j}{p_j - p_a}, \quad (35)$$

and the activation fluid’s own term is

$$W_0 = - \frac{\Delta V_a}{p_a - p_0}, \quad p_0 = P_a(0), \quad (36)$$

which is zero for a perfectly incompressible fluid. Since $\Delta V_a = - \sum_j \Delta V_j$, these definitions satisfy $\sum_j W_j(p_j - p_a) + W_0(p_0 - p_a) = \sum_j \Delta V_j + \Delta V_a = 0$ *identically*. So (34) rebuilt from a point’s own weights reproduces the simulated p_a exactly. The “ p_a from weights” column of the sweep table checks the bookkeeping (paths, signs, volumes), not the model. The weight is undefined where $p_j = p_a$ (0/0), and such points are left out. Only W_j is identified: pressure–volume data cannot separate an effective area A_j from a stiffness K_j in $W_j = A_j/K_j$.

The *tangent* weight is the local compliance $\partial \Delta V_j / \partial p_j$ with p_a held fixed. A unit increase of input pressure p_j adds the load $\mathbf{g}_j$ (the volume gradient of input chamber j). With the activation chamber’s rank-one term removed from the tangent (fixed p_a), and every other sealed chamber following its own law,

$$W_j^{\text{tan}} = -\mathbf{g}_a^{\text{T}} \mathbf{K}_{-a}^{-1} \mathbf{g}_j, \quad \mathbf{K}_{-a} = \mathbf{A} + \sum_{v \neq a} c_v \mathbf{g}_v \mathbf{g}_v^{\text{T}}, \quad (37)$$

which is evaluated with (28). For W_0 the tangent is $-1/P'_a(\Delta V_a)$. A test checks W^{tan} against finite differences (`test_tangent_weight_matches_finite_difference`).

13.3 Sensitivity of the activation pressure

For fixed weights, differentiating (34) with respect to one weight gives

$$S_k = \frac{\partial p_a}{\partial W_k} = \frac{p_k - p_a}{\sum_j W_j + W_0}, \quad (38)$$

where $p_k - p_a$ becomes $p_0 - p_a$ for W_0 . Increasing W_k pulls p_a towards p_k , and $S_k = 0$ where $p_k = p_a$: there, a weight has no influence on the activation pressure, however large or nonlinear it is. This is exactly where the secant weight of a slack membrane is singular, $W \propto |\Delta p|^{-2/3}$. The sweep plots colour every sample of W_k by $|S_k|$, and the fit below uses it as its weighting.

13.4 Piecewise polynomial weights

Each weight is fitted as a polynomial in its own pressure difference, $\Delta p_j = p_j - p_a$ for an input and $\Delta p_0 = p_a - p_0$ for the activation fluid. There is *one polynomial for $\Delta p > 0$ and one for $\Delta p < 0$* , because a membrane deflecting towards the activation chamber and one deflecting away from it can meet different obstacles. Each side is a separate *piece* with its own degree. A weight sampled on one side only uses that side's polynomial for both signs.

Sensitivity-weighted least squares. For one piece with samples $(\Delta p_i, W_i)$ and sensitivities S_i from (38), evaluated with the sample's own weights, the polynomial of degree n minimises

$$\sum_i \left(\frac{|S_i|}{\max_k |S_k|} \right)^2 \left(\hat{W}(\Delta p_i) - W_i \right)^2, \quad \hat{W}(\Delta p) = \sum_{m=0}^n c_m \Delta p^m. \quad (39)$$

By (38), $|S_i| |\hat{W} - W_i|$ is to first order the error in p_a caused by the error in the weight. So the fit minimises the linearised activation-pressure error, not the error in W . Without the weighting, the few samples just off $\Delta p = 0$, where W is huge, dominated the fit and forced high degrees with enormous coefficients, even though they barely affect p_a . For conditioning, the fit is done in $\Delta p / \max |\Delta p|$ and rescaled afterwards. Samples with $|\Delta p| \leq 10^{-6} \max |\Delta p|$ are left out, and the degree is capped at the number of distinct points minus one, and at 10.

Where the weighted average breaks down. Equation (38) requires $\sum_j W_j + W_0$ to be clearly positive. A sample whose weight sum is below 10^{-3} of $\sum_k |W_k|$ is given no sensitivity and is left out of the polynomial fits (it is still part of the tolerance check). The typical cause is an unmodelled bias, which the next subsection removes.

13.5 Bias of non-neutral paths

A closed chamber *inside* a path that is not neutral at rest pushes volume into the activation chamber even when every pressure is equal. Examples are a gas weight chamber filled above ambient, or a liquid one filled with more or less liquid than its volume (a chamber is neutral when its pressure law gives zero gauge pressure at its rest volume). Where all pressures around the activation chamber are equal, p_a then still differs from them, which no weighted average can reproduce. The secant weights $W_j = \Delta V_j / \Delta p_j$ are forced to cancel: in one test design, $-152.6 + 109.7 + 42.6 + 0.3 \approx 0 \text{ mm}^3/\text{kPa}$ at equal input pressures, which made S_k diverge. Such a *biased* path is modelled as

$$\Delta V_j = b_j + W_j(\Delta p_j) \Delta p_j, \quad (40)$$

which puts $B = \sum_j b_j$ in (34). $W_j = (\Delta V_j - b_j) / \Delta p_j$ is then fitted like any other weight.

b_j is *measured*, not fitted. When fitted as a free constant together with the weights, it trades off against the polynomials. The tolerance on p_a cannot tell them apart, and the fitted B varied from 0.6 to 18 mm^3 with the chosen degrees on the test design. Instead, before the sweep, one

extra solve is made at the *neutral state*: every constant-pressure input at the activation chamber's rest pressure p_0 . There an unbiased path pushes nothing, so what a biased path pushes is its bias. The small pressure difference that remains is corrected to first order: $b_j = \Delta V_j - W_j^{\text{tan}} \Delta p_j$. On the test design this gives $b = 5.88 \text{ mm}^3$, and the corrected weights along the sweep become positive and peak at $\Delta p = 0$, as for a slack membrane ($1.75 \dots 15.4 \dots 1.88 \text{ mm}^3/\text{kPa}$), instead of passing through -152.6 . The solve is skipped when no path is biased.

Solving the equation. Given the fitted weights, p_a for an input set is a root of

$$r(p_a) = \sum_k \hat{W}_k(\Delta p_k) (p_k - p_a) + B = 0. \quad (41)$$

Without a bias, a weighted average must lie between the lowest and highest pressure involved, so r is sampled at 121 points on $[\min_k p_k, \max_k p_k]$. A bias can move p_a outside that range, so the interval is then widened by about $B/\sum_k \hat{W}_k(0)$. Every sign change is refined with Brent's method.

Polynomial weights can make (34) *multivalued*: several activation pressures satisfy it for the same inputs. Picking the root closest to the simulated p_a would hide this, since the equation, used on its own, has nothing to pick with. The fit therefore scores every sample by its *worst* root, so an equation only meets the tolerance if every activation pressure it allows does. On the test design, the first version of this search accepted an equation that was within 0.29 kPa at every sample by its nearest root, but 1.2 kPa off by another root.

Choosing the degrees. The user sets a tolerance on the *output* of the equation: the fit must reproduce the simulated activation pressure of every sample within, by default, 1 kPa. The degrees are then made as low as possible. All pieces start at degree 0 (constant weights). Two search strategies are available:

- **Lowest total order** (default): try every combination of piece degrees with total order $0, 1, 2, \dots$ and stop at the first total order at which some combination meets the tolerance, taking the one with the smallest maximum error. This is guaranteed to give the lowest possible total order.
- **Biggest own error first** (greedy): while the tolerance is not met, add one degree to the piece with the largest *own* error. That is the worst p_a error when only that piece uses its polynomial and every other weight takes its sampled value. This is faster, but it can end at a higher total order, because errors of different pieces can cancel.

No piece is raised past the first degree that fits its points exactly. If the tolerance cannot be met, the best combination found is returned and marked as such. The search is also limited to 2000 degree combinations, since an unreachable tolerance would otherwise make the exhaustive search try millions of them. When the budget runs out, the best combination found so far is returned, marked as stopped. The result is written out as LaTeX: (34) with each weight given as a two-case polynomial in its pressure difference, and B with its parts b_j .

Checking between the sweep points. The tolerance is checked at the sweep points only. The application therefore also plots the simulated activation pressure with the fitted equation over it: a curve for a one-input sweep, and a wireframe over the simulated surface for a two-input sweep. The equation is evaluated on a finer grid than the sweep, and any extra roots are shown. A high-degree polynomial can pass through every sweep point and still oscillate between them. That is visible there and is a reason to sweep more densely rather than to tighten the tolerance.

Remark. When the activation fluid is nearly incompressible ($W_0 \ll W_j$), (34) depends only on the *ratios* of the input weights: scaling every W_j by the same factor leaves p_a unchanged. A

tolerance on p_a then cannot pin down the absolute size of the weights, and fitted weights can be off from the sampled ones by a common factor while still meeting the tolerance. The fitted equation is correct as a map from input pressures to p_a . Individual fitted W_j should only be compared with each other.

14 Verification

The repository contains an automated test suite (`python -m pytest tests`). Grouped by what they establish, the tests check that:

- **the discrete system is consistent:** the residual is the gradient of the energy and the tangent is its Jacobian, by finite differences, with active contact, for gas chambers, and for sheet contact;
- **the elements converge to known solutions:** clamped square and circular plates against Kirchhoff plate theory [15]; an inflated circular membrane against Hencky’s solution [8]; bending energy independent of mesh orientation;
- **the chambers behave as intended:** the volume change of an imposed paraboloid; a very stiff sealed chamber gives the same state as a constant-pressure chamber at the same final pressure; a custom pressure law matches the equivalent built-in law; Boyle’s law on the gas share; a gas pocket cannot be squeezed to zero; an under-filled liquid chamber pulls the membrane to its liquid volume;
- **contact is correct:** signed distances match a brute-force search on convex and non-convex shapes; cached queries match the full search; a membrane inflates against a flat obstacle; an inflating sheet lifts another without passing through it;
- **the solver is reproducible:** a warm-started solve reaches the same state as a fresh one;
- **the characterisation is correct:** weights of direct, ambient and bulk-modulus paths rebuild the activation pressure; a stiffer weight fluid transfers more; the tangent weight matches finite differences; the fitted equation uses the lowest degrees that meet the tolerance, with one polynomial per sign; a measured bias enters the equation exactly; a multivalued equation is judged by its worst root; the search stops at its budget and can be cancelled; cancelling weight sums give no sensitivity;
- **the CAD pipeline works:** part names, closed outward meshes, mid-surfaces, measured thickness, chamber–membrane coupling and project save/load on a generated test assembly.

15 Limitations

- **Static only.** There is no inertia, damping or fluid flow, so the switching dynamics of the output stage and the flow through resistors are outside the model.
- **Lumped chambers.** Every chamber has one uniform pressure. Where a chamber covers only part of a membrane, the pressure is still applied to the whole membrane (with a warning).
- **Penalty contact.** Contact allows a small penetration (about 5 % of the thinnest thickness by default) and makes the tangent stiff. There is no friction and no self-contact of a sheet. Sheet contact can still stall under very large pushes (one test of this case does not yet pass).

- **Nearly incompressible chambers** use a stiff linear pressure law rather than a mixed formulation with the pressure as an unknown. Very high stiffness is therefore ill-conditioned, although the second-order correction keeps it usable.
- **Material models.** Only incompressible neo-Hookean and St. Venant–Kirchhoff sheets are available, with no viscoelasticity or damage, and there is no check that stretches stay within the material’s limits.
- **Robustness.** The globalised Newton method has no convergence guarantee. Snap-through and hard contact cases can still stall before full load; the solve then reports the last converged load factor rather than a result at full load.

References

- [1] J. A. Bærentzen and H. Aanæs, “Signed distance computation using the angle weighted pseudonormal,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 11, no. 3, pp. 243–253, 2005.
- [2] J. Bonet and R. D. Wood, *Nonlinear Continuum Mechanics for Finite Element Analysis*, 2nd ed. Cambridge University Press, 2008.
- [3] C. Ericson, *Real-Time Collision Detection*. Morgan Kaufmann, 2004.
- [4] C. Geuzaine and J.-F. Remacle, “Gmsh: a 3-D finite element mesh generator with built-in pre- and post-processing facilities,” *International Journal for Numerical Methods in Engineering*, vol. 79, no. 11, pp. 1309–1331, 2009.
- [5] E. Grinspun, A. N. Hirani, M. Desbrun, and P. Schröder, “Discrete shells,” in *Proc. ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, 2003, pp. 62–67.
- [6] E. Grinspun, Y. Gingold, J. Reisman, and D. Zorin, “Computing discrete shape operators on general meshes,” *Computer Graphics Forum*, vol. 25, no. 3, pp. 547–556, 2006.
- [7] W. W. Hager, “Updating the inverse of a matrix,” *SIAM Review*, vol. 31, no. 2, pp. 221–239, 1989.
- [8] H. Hencky, “Über den Spannungszustand in kreisrunden Platten mit verschwindender Biegesteifigkeit,” *Zeitschrift für Mathematik und Physik*, vol. 63, pp. 311–317, 1915.
- [9] A. Jacobson, L. Kavan, and O. Sorkine-Hornung, “Robust inside-outside segmentation using generalized winding numbers,” *ACM Transactions on Graphics*, vol. 32, no. 4, 2013.
- [10] M. Li, Z. Ferguson, T. Schneider, T. Langlois, D. Zorin, D. Panozzo, C. Jiang, and D. M. Kaufman, “Incremental potential contact: intersection- and inversion-free, large-deformation dynamics,” *ACM Transactions on Graphics*, vol. 39, no. 4, 2020.
- [11] T. Möller and B. Trumbore, “Fast, minimum storage ray-triangle intersection,” *Journal of Graphics Tools*, vol. 2, no. 1, pp. 21–28, 1997.
- [12] L. S. D. Morley, “The constant-moment plate-bending element,” *Journal of Strain Analysis*, vol. 6, no. 1, pp. 20–24, 1971.
- [13] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. Springer, 2006.
- [14] A. Paszke *et al.*, “PyTorch: an imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems 32*, 2019.

- [15] S. Timoshenko and S. Woinowsky-Krieger, *Theory of Plates and Shells*, 2nd ed. McGraw-Hill, 1959.
- [16] A. van Oosterom and J. Strackee, “The solid angle of a plane triangle,” *IEEE Transactions on Biomedical Engineering*, vol. BME-30, no. 2, pp. 125–126, 1983.
- [17] L. Verlet, “Computer ‘experiments’ on classical fluids. I. Thermodynamical properties of Lennard-Jones molecules,” *Physical Review*, vol. 159, no. 1, pp. 98–103, 1967.
- [18] P. Virtanen *et al.*, “SciPy 1.0: fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.